Chapter 13 exercises • I/O systems

Build state/variable tables before consulting the separate answers.

OS13-E01 • PIO publication and lost wakeups

Use the three-register, single-submitter model. The waiter tests done.

1. Give an interleaving exposing old DATA if CPU sets READY first.
2. Give a lost wakeup from checking done before sleeping. Specify the lock's scope.

OS13-E02 • Partial vector writes

Iovec lengths are[120,80,300]. First writev returns 170; a retry on the remainder returns 130.

1. Give base displacements and lengths after each result.
2. If the next result is EAGAIN, should the original vector be resent?

OS13-E03 • Virtual timers

Now4 ms, deadlines[7(A),10(B),10(C)], one one-shot hardware timer.

1. Insert6(D), then cancel A. Trace subsequent hardware deadlines.
2. Must a user thread execute each callback exactly at its deadline?

OS13-E04 • DMA cancellation and reclamation

Request 8 is using DMA. Timeout requests abort, then normal completion and abort acknowledgment arrive in that order. In this model, normal completion or a successful abort acknowledgment guarantees that all DMA for this request has stopped; platform-required synchronization is still needed before unmapping. Each potentially late callback holds a stable request reference before it is queued. If timeout is reported before quiescence, the DMA buffer remains unavailable for reuse until hardware stops and synchronization completes; alternatively DMA can use a kernel-owned bounce buffer.

1. Why cannot timeout immediately unmap/free?
2. Define and prove at-most-once terminal publication.
3. May the winning completion immediately free the request control block?

OS13-E05 • Finite double-buffer batch

Four blocks take 3 ms input and 5 ms processing each. Both buffers start empty. One input device and one CPU can work concurrently on different slots; start each eligible operation as soon as its prerequisite and its server are available. CPU cannot read before input completes and the device cannot overwrite a CPU-owned slot.

1. List input/processing intervals.
2. Compare one/two-buffer total time and the steady-state bottleneck.

OS13-E06 • STREAMS watermarks

An ordinary data queue has capacity 1200 B, initially 600 B, with 300 B messages. Pause upstream at occupancy $\geq$900 B; resume only when occupancy is strictly below 400 B. Two further messages are already waiting in an upstream buffer.

1. After one acceptance stops upstream, when does300 B service back-enable it?
2. With two upstream messages, is total buffering only300 B?

OS13-E07 • Resource bounds and Little's relation

Each request is 8 KiB. Device capacity is 60000/s; one equivalent CPU core supplies 1 CPU-second per second and each request consumes 10 µs. Memory bandwidth is 2 GiB/s with 24 KiB traffic per request. For the same requests and a stable workload, measure from system entry to completion: the completion rate is 50000/s, mean response 400 µs, of which mean service is 100 µs and the rest is waiting. The device permits concurrent service, and service occupancy counts all requests in service.

1. Calculate each resource bound and the system upper bound.
2. Find average requests in system, service, and waiting.
3. With70000 arrivals/s, no dropping, and at most60000 completions/s, can L remain20?

OS13-E08 • Naming, LBA, and durability

Blocks are 4096 B and aligned to 512 B sectors. Read 100 B at offset 12345; logical block 3 maps to partition block 90; partition start LBA is 1024.

1. Find displacement, whole-disk sector and inclusive byte offsets containing this read; separately give the sector range of the whole block.
2. Does buffered write returning 100 prove power-loss safety?
3. If printing finished but acknowledgment was lost, does retrying a host job ID guarantee exactly-once printing?

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 13.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study